Lab1- QuestaSim

This lab session provides a hands-on introduction to QuestaSim. Students will learn how to navigate the tool, create projects, add and compile source files, and run simulations using both the GUI and script-based approaches. The session emphasizes practical skills such as waveform visualization, hierarchical signal exploration, and basic use of Tcl/Do files. ALL SESSIONS IN THIS COURSE WILL BE DEVELOPED IN LINUX.

1. What Is QuestaSim?

QuestaSim is a professional digital design simulation and verification environment developed by Siemens EDA, derived from the widely used ModelSim simulator, originally designed by Mentor Graphics (you will find references to ModelSim in the tool and the manual). It is designed to support the development and verification of complex digital systems written in VHDL, Verilog, and SystemVerilog, providing engineers with a robust platform to test functional correctness before hardware implementation.

One of the key strengths of QuestaSim is its comprehensive support for advanced verification methodologies, including the Universal Verification Methodology (UVM). The tool offers a rich graphical user interface for waveform visualization, hierarchical design browsing, and debugging, as well as scripting capabilities using Tcl/Do files, which allows automated simulation and regression testing.

QuestaSim is widely adopted in both industry and academia, from small-scale designs to large, multi-million-gate systems. Its combination of simulation, waveform analysis, assertions, and coverage capabilities makes it a versatile tool for ensuring design correctness, improving productivity, and reducing the risk of costly hardware errors.

YOU CAN ACCESS QUESTASIM FROM ROOMS A5-S105, A5-S109 AND A5-S111.
IN ADDITION, YOU MAY NOW ALSO USE IT REMOTELY VIA AULAVIRTUAL AT https://aulavirtual.fib.upc.edu.

2. First Steps

Let's check that everything is ready and run a simple example

Check that QuestaSim is installed and accessible

```
$ setenv SALT_LICENSE_SERVER 1717@questa.fib.upc.edu
$ setenv PATH /opt/siemens/questasim/bin:$PATH
```

1) Check that the license server is enabled:

```
$ echo $SALT_LICENSE_SERVER
1717@questa.fib.upc.edu
```

2) Check if the software is accessible:

```
$ which vsim
/opt/siemens/questasim/bin/vsim
```

3) Check the version:

```
$ vsim -version
QuestaSim-64 vsim 2025.1_1 Simulator 2025.02 Feb 13 2025
```

Common binaries installed

The following is a list of the main QuestaSim binaries you will find:

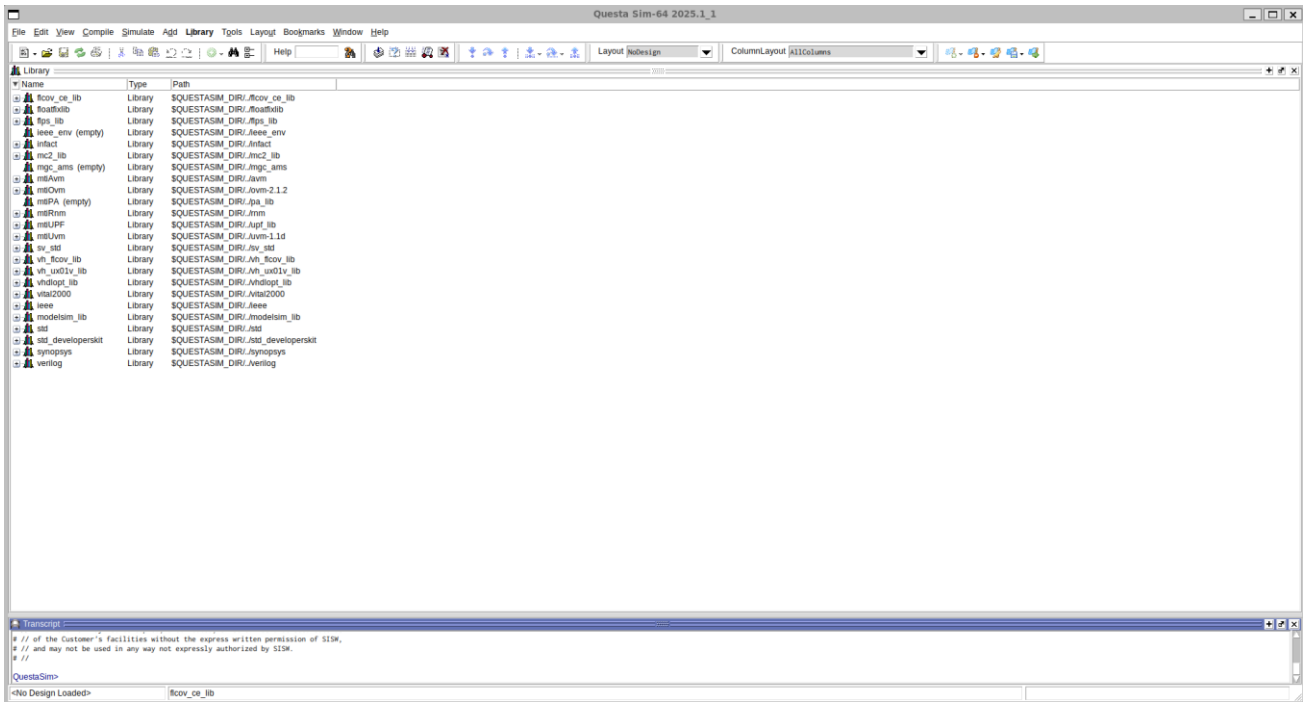
Tool	Description
Core tools	
`vsim`	The simulator (GUI + command line)
`vlib`	Creates a working library for your design
`vlog`	Compiles Verilog/SystemVerilog source
`vcom`	Compiles VHDL source
`vopt`	Optimizer for compiled designs (used for speed/memory improvements)
`vrun`	A batch simulation runner that can execute simulations with command-line options or scripts
`vcover`	Collects and reports code coverage
Library/project management	
`vdir`	Lists design libraries and directories in your working environment.
`vdel`	Deletes compiled design libraries or specific objects from your work library
`vmap`	Maps logical libraries to physical directories
Utilities and helpers	
`qrun`	"Compile & simulate" one-shot wrapper (combines vlog/vcom/vsim)
Debug / GUI-related	
`vis`	Waveform viewer/inspector; similar to the GUI waveform viewer but can be scripted
`visualizer`	Advanced waveform viewer/debugger (depends on edition/license)

QuestaSim GUI

Just run the following command to launch the QuestaSim GUI:

```
$ vsim
```

You will see a window like this:



You can see two sub-windows: Library (on top), and Transcript (on bottom)

- Library: shows all design libraries currently loaded
  - A library is a collection of compiled design units (Verilog modules, VHDL entities)
  - The default working library is called work.
- Transcript: shows the simulation console output, including messages, errors, warnings, and commands executed. Examples of what appears are compilation messages (vlog, vcom) or simulation run output. You can run command from there, but not that it is not a normal Linux shell, but a Tcl command interpreter.

This GUI lets you compile, simulate, and debug your design visually without using the CLI (Command-Line Interface).

On top of the window, you can see the different menus/sub-menus offered by the tool:

- File
- Edit
- View
- Compile
- Simulate
- Add
- Library
- Tools
- Layout
- Bookmarks
- Window
- Help

Navigate through them to see the options the tool offers. Note that new menu entries may appear as you open new functionalities (for instance, when you work with waveforms, you will see a Wave menu entry). Some actions (such as File>Save options) have different meanings depending on which sub-window is active.

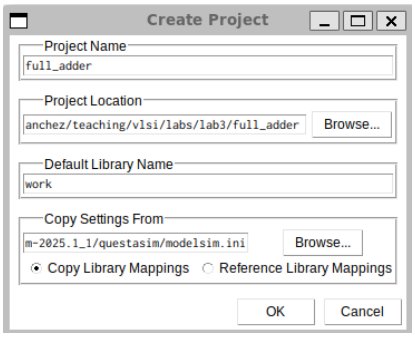
3. My First Design: A 4-bit Full-Adder

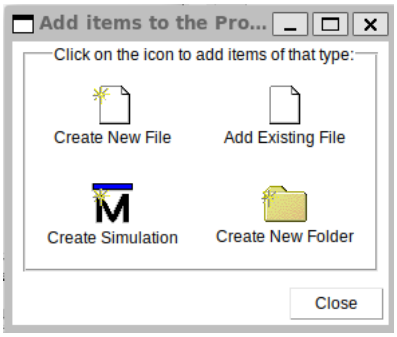
After finishing the Lab2 session, where we designed the physical layout for a 1-bit full-adder, let's continue by creating a 4-bit full-adder as a ripple adder by instantiating copies of the 1-bit full-adder. However, in this session and next ones, we will work at the logic/RTL level instead of the physical design we practiced in Lab1 and Lab2.

Create a directory (for instance at **`$HOME/miri/pv/lab1/full_adder`**) and copy files **`full_adder_1bit.v`**, **`full_adder_4bit.v`** (the sources of the design), and **`tb_full_adder_4bit.v`** (the testbench). You can also create a project manually doing:

```
QuestaSim> project new ./ full_adder
QuestaSim> project addfile full_adder_1bit.v
QuestaSim> project addfile full_adder_4bit.v
QuestaSim> project addfile tb_full_adder_4bit.v
```

You can also perform the previous step by creating a new project (File>New>Project) and writing/adding your source files (see example video "*How to use Questasim?*" at YouTube). Using a project has the advantage that you can later re-open it easily.





Edit and review the sources to understand their contents and purpose. *You can use any editor of your preference, I personally like VSCode. If this is also your case, install an Extension to highlight the syntax for Verilog/SystemVerilog sources (for instance, I use Verilog-HDL/SystemVerilog/Bluespec System Verilog, by Masahiro Hiramori). Other editors (such as gvim) may have the option to enable syntax highlighting as well. QuestaSim also contains an editor you can use.*

In the Transcript window (recall that this is a Tcl interpreter, not a Linux shell. For instance, Bash environment variables are not directly accessible), type the following commands to create a working directory in for example directory and compile the sources (using vlog command):

```
QuestaSim> cd ~/miri/pv/lab1/full_adder
QuestaSim> vlib work ← You don't need that if you created as a project
QuestaSim> vlog +acc full_adder_1bit.v
QuestaSim> vlog +acc full_adder_4bit.v
QuestaSim> vlog +acc tb_full_adder_4bit.v
```

The "-acc" (access visibility) argument in compilation lines is important for expanding the hierarchy and viewing waveforms. Without it, the results are optimized and signals may not be visible – similar to using -O0 in gcc compilation)

In the Library window you will see a new entry with your work library. If you expand it, you will see the three sources. A work directory is also created in **`$HOME/miri/pv/lab1/full_adder`**. Now, start the simulation of the testbench as:

```
QuestaSim> vsim work.tb_full_adder_4bit
```

You will see a new set of sub-windows open under tab sim:

- Instance window shows the hierarchical structure of your design (like a tree)
- Objects window shows all the signals, ports, and variables in the currently selected instance
- Processes window shows the active processes (always blocks, initial blocks, continuous assignments) running in the current instance

In the Instance window you can now expand your design instances (by clicking the + symbol or select and instance at right-click + Expand Selected). You should see uut (your DUT instantiated in the testbench) and fa[0-3] for the full_adder_1bit instances in the full_adder_4bit module. If you right double-click the tb_full_adder_4bit instance and editor with the source code of the testbench is open as well.

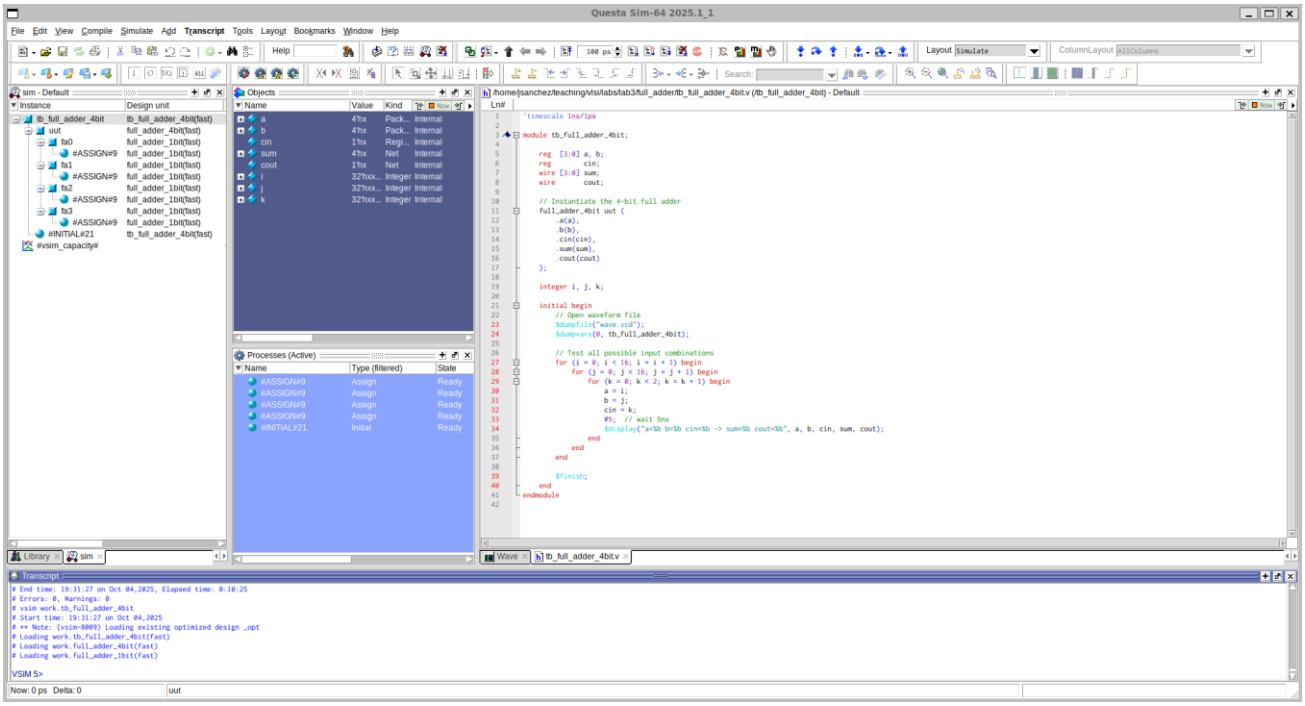
In the Objects window you will see the signals used in the selected instance. Click on the different instances to see the local signals used by each.

Before running the simulation, let's add a Wave window. We can do that by clicking on View>Wave (look at the new tab next to the editor of the testbench) and add signals by doing right-click to a given signal in Object and Add Wave. An alternative to all signals at once in by typing the following in the Transcript window:

```
QuestaSim> add wave *
```

This will acc all signals in the selected instance. You can add recursively all signals in the instance and its submodules by doing "`add wave -r all`" or "`add wave /tb_full_adder_4bit/uut/fa1/*`" for a given instance, though you may find this maybe quite exhaustive. Over time, you will acquire the ability to selectively include only the signals of interest in the waveforms.

At this point, right before running the simulation, you should have something like that:



Now we can run the testbench. For that, we can use the run command in Transcript:

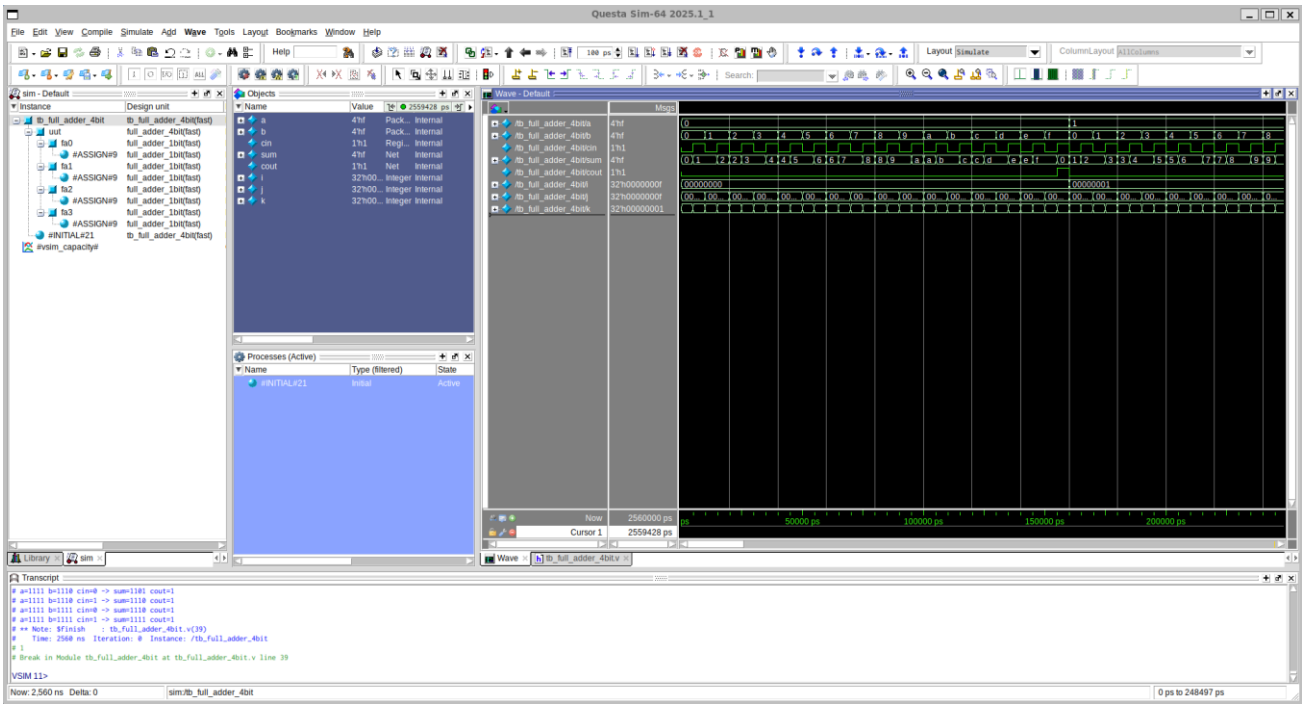
```
QuestaSim> run 500ns
```

or

```
QuestaSim> run -all
```

The first one, as noted, will advance 500ns (or whatever you indicate) on your design. The second will run until it finishes naturally with a **`$finish`** or **`$stop`** statement in your testbench (you will get a pop-up window saying "Are you sure you want to finish?"; click "No" to avoid exiting QuestaSim). Now, if you click in the Wave tab you can see the signals you added. With CTRL + Mouse Scroll-up (or down) you can easily zoom in/out in the waves.

Note that your waveform will contain data only for the signals you selected initially. If you add new signals, they will appear empty in the waveform. In that case, you will need to re-run the simulation.



Additionally, in the Transcript window we can see the output of the run coming from the $display statements in the tb_full_adder_4bit testbench. Note that this testbench is very exhaustive, since it tries all the possible combinations for the DUT inputs (a, b, and cin).

You can also do previous steps by creating a do file with the sequence of commands and run it as … (see example same previous video). For instance, you could have a file run.do with the following content:

```
$ cat run.do
vlog +acc full_adder_1bit.v full_adder_4bit.v tb_full_adder_4bit.v
vsim work.tb_full_adder_4bit
add wave *
run 500ns
```

And then do:

```
QuestaSim> do run.do
```

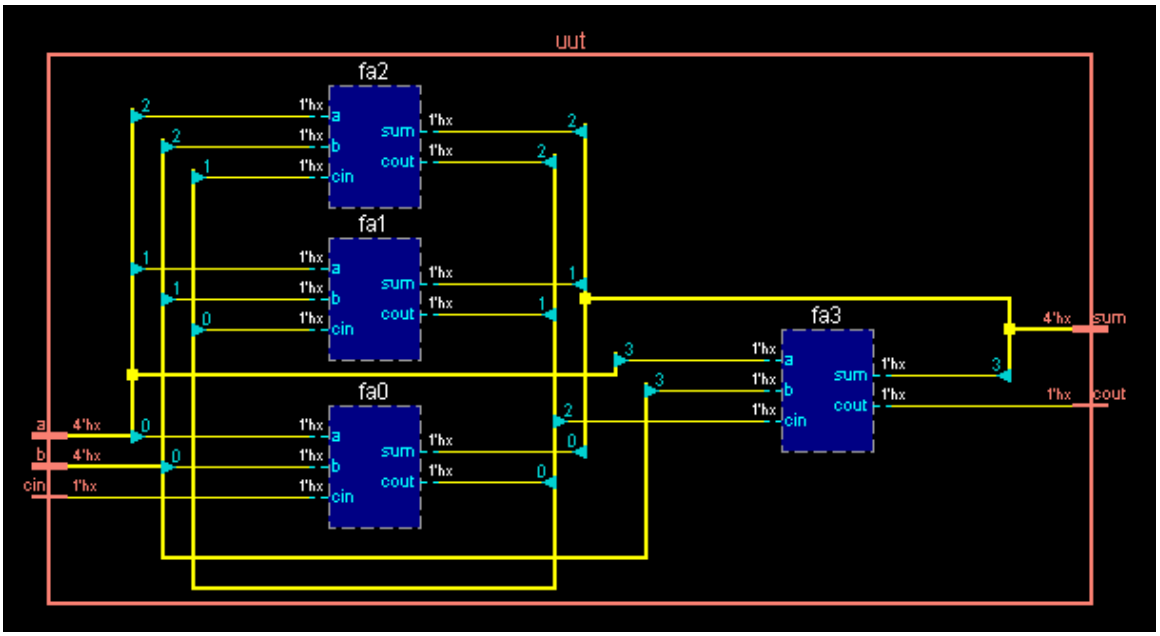
If you look in the directory where you set your project, you will find some new files:

- **`modelsim.ini`**: main configuration file. It defines how the simulator behaves and where it finds your design libraries, compilation settings, and simulation defaults.
- **`transcript`**: just the track of the commands/outputs in your Transcript window.
- **`full_adder.mpf`** (ModelSim/QuestaSim Project File): this is the project file
- **`full_adder.cr.mti`** (Crash Recovery): automatically generated by QuestaSim in the simulator unexpectedly terminates.
- **`work`**: contains the compiled library directory
- **`vsim.wlf`** (Wave Log Format file): used to save your Questa Sim simulation waveforms. You can re-open it later with vsim -view vsim.wlf. This can be very useful when you find a bug and want to report it to the designer for a fix.
- **`wave.vcd`** (Value Change Dump): same information as vsim.wlf, but in a text-based, tool-independent, IEEE Verilog standard format. You may find this format interesting for data analysis using, for instance, Bash or Python scripts.

You may find it very useful to save the waveform layout (not the waveforms, but the signals, with their highlights, grouping, etc.). You can do that just click in the Wave window and clicking the symbol or in File>Save Format … (CTRL-S). You will see a new file named wave.do (or the name you chose). You can easily pass the file to a colleague and use the same signals in your analysis/debug.

Look at the content of wave.do text file from your Linux console, because many times it is easier to add and organize signals just by editing that file than from the GUI tool.

Finally, you may also find it interesting (even familiar) to have a look at the schematic view of your design. For that, you need to run vsim with option -debugDB and just open the Schematic window as View>Schematic (it may take some time to open) and then click View:Full from the top-left corner. You can then click on the modules to expand it.



4. Try Yourself

Here a some follow on activities suggested:

a) It is quite common to compile and run programs from the command line, whereas the GUI is typically used for waveform and coverage analysis. Try using only command-line commands to create a library, compile a design or testbench, and run it.
b) Select design examples completed in MIRI-PD or other (alternatively, you may find basic designs from the internet), and proceed to develop a simple, monolithic testbench for simulation purposes. See several examples at https://www.chipverify.com/verilog/verilog-examples. Each includes a testbench—try writing your own before reviewing the provided solutions.

5. Additional Material

YouTube

QuestaSim

- How to use Questasim?
- How to use Questasim for Beginners | Schematic View | TestBench

Verilog Design

- Verilog Coding with HDLbits (33 lessons, ~15-20 mins each)

How to Practice from Home

EDA Playground at https://www.edaplayground.com

Very useful, free, and easy to use, full of simple and complex examples. You will see designs and testbenches, and you can edit and run them. It has quite a lot limitations compared to QuestaSim, but it is a good starting point.